

3. AI Large Model Voice Interaction

3. AI Large Model Voice Interaction

1. Concept Introduction
 - 1.1 What is "AI Large Model Voice Interaction"?
 - 1.2 Brief Description of Implementation Principles
2. Code Analysis
 - Key Code
 1. Voice Input Node (`1argemodel/1argemodel/asr.py`)
 2. AI Service and Voice Output Node (`1argemodel/1argemodel/model_service.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring Offline Voice Interaction
 - 3.2 Startup and Testing

1. Concept Introduction

1.1 What is "AI Large Model Voice Interaction"?

In the `1argemodel` project, **AI Large Model Voice Interaction** connects the previously introduced **Offline ASR** and **Offline TTS** with the **Large Language Model (LLM)** core, forming a complete conversational system that can listen, speak, and think.

This is no longer an isolated function, but the prototype of a true **voice assistant**. Users can have natural language conversations with the robot through voice, and the robot can understand questions, think about answers, and respond with voice. The entire process is completed locally without requiring network access.

The core of this function is the `model_service` **ROS2 node**. It acts as the brain and neural center, subscribing to ASR recognition results, invoking the LLM for thinking, and then publishing the LLM's text replies to the TTS node for speech synthesis.

1.2 Brief Description of Implementation Principles

The implementation principle of this function is a classic data pipeline:

1. **Voice -> Text (ASR)**: The `asr` node continuously listens to ambient sounds. Once it detects that the user has finished speaking, it converts the audio to text and publishes it to the `/asr_text` topic.
2. **Text -> Thinking -> Text (LLM)**: The `model_service` node subscribes to the `/asr_text` topic. When it receives text from ASR, it sends it as a prompt to the locally deployed large language model (such as `qwen` running through `ollama`). The LLM generates a response text based on the context.
3. **Text -> Voice (TTS)**: After the `model_service` node gets the LLM's response, it publishes it to the `/tts_text` topic.
4. **Voice Playback**: The `tts_only` node subscribes to the `/tts_text` topic. Upon receiving the text, it immediately invokes the offline TTS model to synthesize it into audio and plays it through the speaker.

This process forms a complete closed loop of **Voice Input -> Text Processing -> Text Output -> Voice Output**.

2. Code Analysis

Key Code

1. Voice Input Node (largemodel/largemodel/asr.py)

```
# From largemodel/largemodel/asr.py
class ASRNode(Node):
    def __init__(self):
        # ...
        self.asr_pub = self.create_publisher(String, "asr", 5)
        # ...

    def kws_handler(self)->None:
        if self.listen_for_speech(self.mic_index):
            asr_text = self.ASR_conversion(self.user_speechdir)
            if asr_text != 'error':
                self.asr_pub_result(asr_text)

    def asr_pub_result(self, asr_result:str)->None:
        msg=String(data=asr_result)
        self.asr_pub.publish(msg)
```

2. AI Service and Voice Output Node

(largemodel/largemodel/model_service.py)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.asrsub = self.create_subscription(String, 'asr',
self.asr_callback,1)
        # ...

    def asr_callback(self,msg):
        """Callback function for handling ASR messages."""
        # ...
        result = self.model_client.infer_with_text(msg.data,
message=messages_to_use)
        self.process_model_result(result)

    def process_model_result(self, result, from_seewhat=False):
        """Process the result returned by the model."""
        # ...
        user_friendly_response = response_json.get("response", "我正在处理...")
        # ...
        self._safe_play_audio(user_friendly_response)
        # ...
        self.execute_tools(tools_list)

    def _safe_play_audio(self, text_to_speak: str):
        """
        Synthesizes and plays all non-empty messages only in non-text chat mode.
        """
```

```

        if not self.text_chat_mode and text_to_speak:
            try:
                self.model_client.voice_synthesis(text_to_speak,
self.tts_out_path)
                self.play_audio_async(self.tts_out_path)
            except Exception as e:
                self.get_logger().error(f"Safe audio playback failed: {e}")

```

Code Analysis

The voice interaction functionality of the AI large model is collaboratively completed by two independent ROS nodes, `asr.py` and `model_service.py`, which communicate through the ROS topic `/asr` to form a complete processing loop.

1. Voice Input and Publishing (`asr.py`):

- The `ASRNode` node is responsible for processing voice input. In the `kws_handler` function, it completes recording and text conversion.
- After successful conversion, the `asr_pub_result` function is called. This function places the obtained text string into a `std_msgs.msg.String` message and publishes it to the `/asr` topic through the `self.asr_pub` publisher.

2. Subscription and Response (`model_service.py`):

- During initialization, the `LargeModelService` node subscribes to the `/asr` topic through the `create_subscription` method, specifying `asr_callback` as its callback function.
- When `ASRNode` publishes a message on the `/asr` topic, the ROS system automatically calls `LargeModelService`'s `asr_callback` function, passing the message as the parameter `msg`.

3. Interaction Loop:

- After the `asr_callback` function receives the message, it extracts the text data (`msg.data`) and passes it to the large model for inference (`self.model_client.infer_with_text`).
- After inference is complete, the `process_model_result` function is called. This function extracts the response text from the model results.
- Then, the `_safe_play_audio` function is called, which uses TTS functionality (`self.model_client.voice_synthesis`) to synthesize the response text into audio and play it.
- At this point, a complete interaction loop from "receiving user voice" to "responding to user with voice" is completed. The two nodes are decoupled through ROS's publish-subscribe pattern, with `asr.py` handling input and `model_service.py` handling processing and output.

3. Practical Operations

3.1 Configuring Offline Voice Interaction

To implement a completely offline voice interaction system, you need to ensure that ASR, TTS, and LLM are all configured for offline mode.

1. Open the main configuration file `yahboom.yaml`:

```

gedit ~/yahboom_ws/src/largemodel/config/yahboom.yaml

```

2. Modify/confirm the following key configurations:

```
asr:
  ros__parameters:
    use_online_asr: False          # Critical: Set to False to enable
    offline ASR

model_service:
  ros__parameters:
    use_offline_tts: False        # Critical: Set to False to enable
    offline TTS
    llm_platform: 'ollama'       # Critical: Set to 'ollama' to
    enable offline LLM
```

3. Open the model interface configuration file `large_model_interface.yaml`:

```
gedit ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

4. Confirm all offline model paths are correct:

```
# large_model_interface.yaml

## Offline Large Model
ollama_model: "qwen2.5:7b" # Ensure this model has been downloaded via
ollama pull

## Offline Speech Recognition
local_asr_model: "/path/to/your/SenseVoiceSmall" # Ensure ASR model path is
correct

## Offline Speech Synthesis
zh_tts_model: "/path/to/your/zh_CN-huayan-medium.onnx" # Ensure TTS model
path is correct
# ...
```

3.2 Startup and Testing

1. Start the `largemodel` main program:

Run the following command to start voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

2. Test:

- **Wake up:** Say to the microphone: "Hello, yahboom."
- **Conversation:** After the speaker responds, you can speak your question.
- **Observe logs:** In the terminal running the `launch` file, you should see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, invokes the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the robot answer your question in synthesized voice from the speaker.

Possible issue: Hello yahboom can wake up but audio cannot be captured

Solution:

- Confirm USB microphone is recognized by the system

```
arecord -l
```

Check if "USB Audio Device" appears in the output

- Confirm mic_index configuration

```
grep mic_index  
~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

Recommended to use 0 (uses PulseAudio default source, automatic resampling)

- Confirm PulseAudio default source is USB microphone

```
pactl get-default-source
```

If not, first check the source name:

```
pactl list short sources | grep usb
```

Then set the default source:

```
pactl set-default-source <source_name>
```

- Start testing

```
ros2 launch largemodel largemodel_control.launch.py
```